

Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль



F.CSM301 Алгоритмын шинжилгээ ба зохиомж

Бичвэр жигдлэх программ

Бие даалт №2

Шалгасан багш:

Д.Батмөнх /F.CS21/

Гүйцэтгэсэн оюутан:

Л.Ирмүүн /B232270021/

Улаанбаатар хот
2025 он

АГУУЛГА

1. Бие даалтын зорилго	4
1.1 Программын зорилго.....	4
2. алгоритмын хэрэглээ.....	4
2.1 Greedy алгоритм.....	4
2.2 Dynamic Programming (DP) алгоритм	5
2.3 Зам тооцоолох тохиромжтой алгоритм	5
3. Программын ажиллагаа ба хэрэгжүүлэлт	6
3.1 Төслийн ерөнхий бүтэц.....	6
3.2 Ашигласан технологи ба сангууд	8
3.3 Бичвэр жигдлэх — текст боловсруулалт.....	8
3.3.1 Өгөгдөл Уншилт.....	8
3.3.2 Мөр таслах	8
3.3.3 Мөр форматлах.....	9
3.3.4 Жигдрүүлсэн бичвэр үүсгэх.....	9
4. Console-ийн гаралт	9
4.1 Java хувилбарын гаралт	9
4.2 Python хувилбарын гаралт	10
4.3 Алгоритмын гүйцэтгэлийн мэдээлэл.....	10
4.4 Алгоритмын гүйцэтгэлийн мэдээлэл.....	11
5. Алгоритмын гүйцэтгэлийн шинжилгээ.....	11
5.1 Туршилтын орчны мэдээлэл.....	11
5.2 Туршилтын үр дүнгийн хүснэгт.....	12
5.3 Харьцуулсан шинжилгээ.....	13
5.3.1 Greedy vs DP (адил хэл дээр).....	13
5.3.2 Java vs Python (адил алгоритм дээр).....	13
6. Тест ба баталгаажуулалт.....	14
6.1 Зорилго	14
6.2 JustifyTest.java — Нэгж тестийн баталгаа	14
6.3 test_justify.py — Нэгж тест ба онолын баталгаа	15
6.3.1 Үндсэн бүтэц ба нийтлэг шалгалтууд	15
6.3.2 Монгол текст ба badness-ийн харьцуулалт	15
6.3.3 Захын тохиолдлууд	16
7. Дүгнэлт.....	16

8.	НОм зүй.....	17
----	--------------	----

1. БИЕ ДААЛТЫН ЗОРИЛГО

1.1 Программын зорилго

Энэхүү бие даалтын зорилго нь өгөгдсөн бичвэрийг мөрийн уртад тохируулан зүүн болон баруун талаараа жигдлэх алгоритмын хэрэгжилтийг судалж, Greedy болон Dynamic Programming аргачлалуудыг ашиглан текстийг форматлах системийг боловсруулахад оршино. Жигдлэх үйлдэл нь офисын программуудын бичвэрийг хэлбэржүүлэх үндсэн үйлдлийн нэг тул тухайн алгоритмуудын онолын зарчим, гүйцэтгэлийн ялгаа, зохимжтой хэрэглээг бодит жишээнд туршиж тодорхойлох нь энэхүү даалгаврын гол зорилт юм.

Туршилтын хүрээнд хоёр төрлийн алгоритмаар бичвэрийг мөр мөрөөр хувааж, хоорондын хоосон зайг нөхөн баруун ирмэгийг тэгшлэх үйлдлийг хэрэгжүүлнэ. Мөн эдгээр алгоритмуудын цаг хугацааны төвөгшил, санах ойн зарцуулалт, үнэн зөв ажиллагааг жишээ текстүүд дээр шалгаж, loop invariant болон математик индукц ашиглан онолын хувьд баталгаажуулна. Ингэснээр бичвэрийг жигдрүүлэх бодит хэрэглээнд аль алгоритм илүү тохиромжтой болохыг тогтоох боломж бүрдэнэ.

2. АЛГОРИТМЫН ХЭРЭГЛЭЭ

2.1 Greedy алгоритм

Үндсэн ажиллах зарчим:

Greedy буюу “хомхойлох” алгоритм нь текстийг жигдрүүлэхдээ мөрөнд багтах боломжтой хамгийн олон үгийг шууд сонгон байрлуулж, илүүдэл зайг тухайн мөрөнд жигд хуваарилдаг. Өөрөөр хэлбэл, тухайн үеийн нөхцөлд хамгийн ашигтай сонголтыг хийснээр нийт бичвэрийн хэлбэрийг хурдан бүрдүүлэх зарчмаар ажилладаг.

Greedy арга нь дараагийн мөрийн чанарыг бүрэн тооцдоггүй учир оновчтой шийдлийг баталгаатай олдоггүй боловч гүйцэтгэлийн хурд өндөр, хэрэгжүүлэхэд энгийн байдаг. Мөрийн дундах зайгаар жигд хуваарилалт хийхдээ нийт илүүдэл null character буюу space-ийг (хоосон зай) үг хооронд жигд тараан байрлуулж, үлдсэн зайг зүүнээс баруун тийш нэмэгдүүлэх байдлаар ажиллана.

Ажиллах алхмууд:

1. Нэг мөрөнд багтах боломжит хамгийн олон үгийг сонгоно.
2. Сонгосон үгсийн нийт уртыг мөрийн дээд уртаас хасаж, дутуу зайг тооцно.
3. Дутуу зайг үг хооронд тэнцүү хувааж, шаардлагатай бол үлдэгдлийг зүүнээс эхлэн нэмж байрлуулна.
4. Мөрийг бэлэн болгосны дараа дараагийн мөрөнд шилжинэ.
5. Бичвэр дуусах хүртэл 1–4 алхмыг давтана.

Давуу тал:

- Гүйцэтгэлийн хурд хамгийн сайн.
- Хэрэгжүүлэх логик энгийн.

- Жижиг хэмжээний бичвэр эсвэл стандарт formatting-т сайн ажилладаг.
- Гүйцэтгэлийн хугацаа: $O(n)$

Сул тал:

- Оновчтой текст жигдлэлт баталгаатай биш.
- Зарим мөр бусад мөрнөөс хэт ялгаатай харагдах эрстэлтэй.

2.2 Dynamic Programming (DP) алгоритм

Үндсэн ажиллах зарчим:

Dynamic Programming нь бичвэрийн мөр бүрийн таслалт, хоосон зай хуваарилалтын чанарыг “өртөг” гэж үзэн, нийт бичвэрт хамгийн бага зардалтай жигдрэлтийн байдлыг бодож олдог. Энэ нь Greedy аргын адил тухайн үеийн хамгийн сайн сонголтыг бус, харин бичвэрийн төгсгөл хүртэлх бүх боломжит таслалтыг харьцуулж хамгийн оновчтой хувилбарыг сонгодог учир үр дүн илүү тогтвортой, мэргэжлийн төвшний жигдрэлт гардаг.

DP алгоритм нь мөр бүрийн “муу зай” буюу хоосон зайны тархалтыг квадрат зэрэгтэйгээр зардал болгон тооцож, тухайн мөрийн төгсгөлд үүссэн хоосон зайг хамгийн бага хэмжээнд барина. Ингэснээр эцсийн нийт formatting хамгийн үзэмжтэй болно.

Ажиллах алхмууд:

1. Бүх боломжит таслалтыг судалж мөр бүрийн зардлыг тооцно.
2. $DP[n]$ нь n -эр үгээс эхлэн бичвэрийн төгсгөл хүртэлх хамгийн бага зардлыг илэрхийлнэ.
3. DP хүснэгтийг баруунаас зүүн тийш тооцож бөглөнө.
4. DP хүснэгтийг ашиглан хамгийн оновчтой мөрийн таслалтыг сэргээнэ.
5. Мөр бүрийн хоосон зайг жигд хуваан форматална.

Давуу тал:

- Бичвэр жигдрэлтийн хамгийн оновчтой шийдлийг баталгаатай олно.
- Мөрүүдийн хооронд жигд, үзэмжтэй хэлбэр бүрдүүлдэг.
- Хэл шинжлэл, хэвлэлийн системүүдийн төвшинд өргөн хэрэглэгддэг.
- Гүйцэтгэлийн хугацаа: $O(n^2)$ — цаг хугацааны төвөгшил их.

Сул тал:

- Greedy-тай харьцуулахад цаг хугацааны төвөгшил их.
- Илүү их санах ой шаарддаг.
- Хэрэгжүүлэх логик илүү төвөгтэй.

2.3 Зам тооцоолох тохиромжтой алгоритм

Даалгаврын нөхцөлөөс шалтгаалан аль алгоритм илүү тохиромжтой болохыг дараах байдлаар авч үзье:

1. Хурд хамгийн чухал, бодолт олон давтагддаг ердийн жигдрэлт

2. Хэвлэлийн чанарын өндөр шаардлагатай, үзэмж чухал жигдрэлт
3. Хоёрын дундах, чанар ба хурдыг зэрэг харгалзсан хосолмол нөхцөл

Дээрх гурван нөхцөлийг хангахад аль алгоритм нь тохиромжтой байгааг доорх хүснэгтэд харуулав.

Тооцоолох зам	Алгоритм	Шалтгаан
1. Хурдан, олон удаагийн жигдрэлт	Greedy алгоритм	Мөрт багтах үгсийг шууд сонгон байрлуулдаг тул цаг хугацааны төвөгшил бага ($O(n)$), их хэмжээний бичвэрийг богино хугацаанд боловсруулахад тохиромжтой.
2. Өндөр чанарын, үзэмжтэй жигдрэлт	Dynamic Programming алгоритм	Мөр бүрийн хоосон зайн “өртөг”-ийг тооцож, нийт зардлыг хамгийн бага байхаар бүх боломжит таслалтыг харьцуулдаг тул хамгийн жигд, мэргэжлийн төвшний хэлбэржүүлэлт гардаг.
3. Хосолмол нөхцөл (зарим хэсэгт хурд, зарим хэсэгт чанар чухал)	DP + Greedy хосолсон хэрэглээ	Хурд шаарддаг хэсэгт Greedy-г, чанарын өндөр шаардлагатай гарчиг, гол текстэд DP-г ашигласнаар нөөцийг хэмнэж, чанарыг алдагдуулахгүй байх боломжтой.

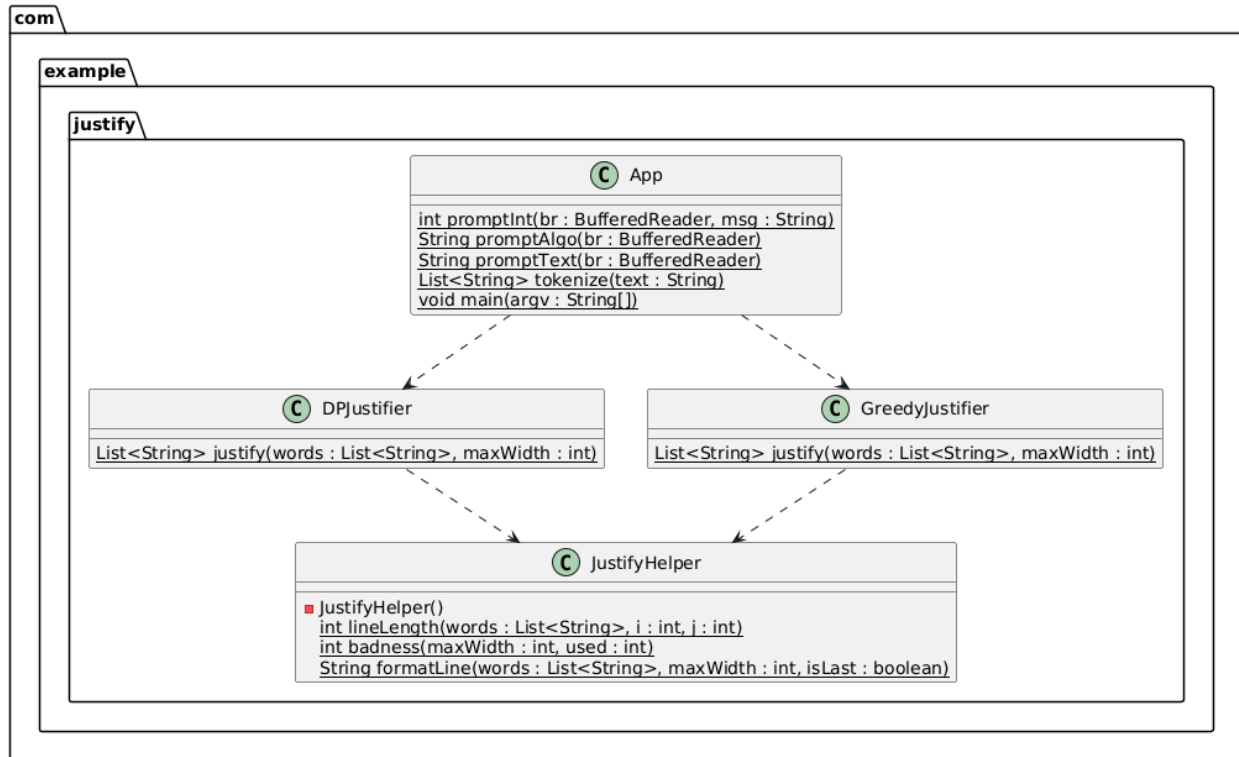
Хүснэгт 1. Бичвэр жигдлэхэд тохирох алгоритмууд

3. ПРОГРАММЫН АЖИЛЛАГАА БА ХЭРЭГЖҮҮЛЭЛТ

3.1 Төслийн ерөнхий бүтэц

Энэхүү бие даалтын зорилго нь өгөгдсөн бичвэр болон мөрийн дээд уртыг ашиглан Greedy болон Dynamic Programming алгоритмаар бичвэрийг мөр мөрөөр хувааж, баруун ба зүүн талаараа жигдэлсэн текст гаргах консоль программ хэрэгжүүлэхэд оршино.

Төслийг хоёр хэл дээр (Java, Python) хэрэгжүүлсэн бөгөөд текст жигдрүүлэх үндсэн логикийг дахин ашиглах боломжтой модулиуд болгон салгаж, алгоритм тус бүрийг тусдаа класс/модульд байрлуулсан.



Зураг 1. Төслийн бүтэц

Класс	Үүрэг, зориулалт
App	Программын үндсэн урсгалыг удирдах класс. Хэрэглэгчээс оролт авах (мөрийн өргөн, алгоритм сонголт, текст), үгсийг салгаж боловсруулах, Greedy эсвэл DP алгоритмын аль нэгийг ажиллуулж жигдлэгдсэн бичвэр үүсгэнэ. Мөн алгоритмын гүйцэтгэлийн хугацаа, санах ойн хэрэглээг хэмжиж харьцуулж хэвлэнэ.
DPJustifier	Dynamic Programming алгоритмаар бичвэр жигдрүүлэх үндсэн хэрэгжилт. Мөр бүрийн таслалтыг бүх боломжит хувилбараар тооцож, badness (хоосон зайны зардал) хамгийн бага байх байгууламжийг сонгодог. DP хүснэгт (cost[], nextBreak[]) ашиглан таслалтын оновчтой дарааллыг сэргээнэ. Мөр бүрийг форматлахдаа JustifyHelper-ийн туслах функцүүдийг ашиглана.
GreedyJustifier	Greedy зарчмаар бичвэр жигдрүүлэх хэрэгжилт. Нэг мөрөнд багтах боломжтой хамгийн олон үгийг шууд сонгон байрлуулж, хоосон зайг жигд хуваарилна. Жигдрүүлэх үйлдэл хурдан бөгөөд DP-с хөнгөн. Мөр форматлах ажлыг JustifyHelper ашиглан гүйцэтгэнэ.
JustifyHelper	Бүх нийтлэг форматлах үйлдлүүдийг нэгтгэсэн туслах класс. Мөрийн нийт урт (lineLength), хоосон зайны зардал (badness), үг хооронд зай хуваах (formatLine) зэрэг Greedy болон DP

	алгоритмын хоёулд ашиглагдах функцүүдийг агуулна. Utility класс тул объект үүсгэхээс сэргийлж private constructor ашигладаг.
--	--

Хүснэгт 2. Төслийн бүтцийн тайлбар

3.2 Ашигласан технологи ба сангууд

Технологи ба сангууд	Үүрэг
Java	Текст жигдрүүлэх үндсэн логикийг хэрэгжүүлсэн гол хэл; Greedy ба DP алгоритмын Java хувилбар, гүйцэтгэлийн хэмжилт, нэгж тестүүд.
Python	Алгоритмуудын хоёр дахь хувилбарыг хэрэгжүүлж, хэл хоорондын гүйцэтгэлийг харьцуулахад ашигласан; хурдан туршилт, прототип бичихэд тохиромжтой.
JUnit / pytest	Greedy болон DP алгоритмуудын нэгж тест, инвариант ба индукцийн баталгааны тестүүдийг бичихэд ашигласан.
VSCoде / IntelliJ IDEA / PyCharm	Хөгжүүлэлтийн орчин; Java ба Python код бичих, тест ажиллуулах, де баг хийхэд хэрэглэсэн IDE-үүд.

Хүснэгт 3. Технологи ба сангууд

Хэрэглэсэн дээрх технологи, сангуудын тусламжтайгаар бичвэрийг хоёр өөр алгоритмаар жигдрүүлэн форматлах, тэдгээрийн гүйцэтгэл болон нөөцийн хэрэглээг хэмжиж харьцуулах боломжтой болсон.

3.3 Бичвэр жигдлэх — текст боловсруулалт

Энэ хэсэгт бичвэрийг унших, үгсийг боловсруулах, мөр таслах, жигдрүүлэх алгоритмд бэлтгэх гол шат дамжлагыг товчлон тайлбарлав.

3.3.1 Өгөгдөл Уншилт

1. Хэрэглэгчээс мөрийн өргөн, бичвэрийг **App** класс ашиглан уншина.
2. Уншсан бичвэрийг whitespace-аар салгаж **tokenize()** функцээр үгсийн жагсаалт үүсгэнэ.
3. Үгсийн жагсаалт Greedy болон DP алгоритмд шууд ашиглагдах үндсэн оролт болно.
4. Үг бүрийн уртыг тооцож, мөрөнд багтах боломжийг **JustifyHelper.lineLength()** функцээр шалгана.
5. Greedy алгоритм нэг мөрөнд багтах хамгийн олон үгийг урьдчилан сонгоно.
6. DP алгоритм бүх боломжит таслалтыг шалгаж зардлын хүснэгт (cost) бэлтгэнэ.

3.3.2 Мөр таслах

7. GreedyJustifier: Мөрөнд багтах хүртэл үгсийг дарааллаар нэмээд багтах боломжгүй үед тухайн хэсгийг нэг мөр болгоно.

8. DPJustifier: Хоосон зайны “зардал”-ийг тооцож хамгийн бага нийт зардалтай таслалт сонгоно.

3.3.3 Мөр форматлах

9. Форматлах ажиллагаа нь JustifyHelper.formatLine() функцээр хийгдэнэ.
10. Сүүлийн мөр эсвэл ганц үгтэй мөрийг зүүн талд байрлуулна.
11. Бусад мөрөнд хоосон зайг slot-удад жигд хуваарилж баруун тийш тэгшилнэ.
12. Зам сэргээх: reconstruct функц ашиглаж бүх алгоритм prev хүснэгтээр зорилтоос эхлэл уруу ухарч, дарааллаар эргүүлэн маршрут гаргаж авна.

3.3.4 Жигдрүүлсэн бичвэр үүсгэх

13. Алгоритмаас буусан мөрүүдийг дарааллаар нь нэгтгэж List<String> хэлбэрээр буцаана.
14. App класс гаралтыг console дээр хэвлэж, ажилласан хугацаа ба санах ойг хэмжинэ.

4. CONSOLE-ИЙН ГАРАЛТ

Энэхүү хэсэгт жигдрүүлэх программын Java болон Python хувилбаруудын console дээрх гаралтыг харуулав. Хоёр хувилбар ижил логиктой ажиллаж, хэрэглэгчээс мөрийн өргөн, алгоритмын төрөл, бичвэрийн агуулгыг авч жигдлүүлсэн үр дүнг хэвлэнэ.

4.1 Java хувилбарын гаралт

Зураг 2-т Java хэл дээрх консоль программын ажиллаж буй байдал үзүүлэв.

1. Хэрэглэгчээс:
 - Row width: 30
 - алгоритмын сонголт (1 → DP, 2 → greedy, жишээ зурагт 1)
 - олон мөр текст (Монгол үлгэрийн эх)
 - хоосон мөрөөр дуусгасан.
2. DPJustifier алгоритм текстийг мөрөнд хувааж, зүүн ба баруун талаараа тэгшлэн хэвлэж байна.
3. Доод хэсэгт:
 - Execution time: 2.992 ms
 - Used memory: 68.6 KiBгэж гарах бөгөөд энэ нь Java хувилбарын гүйцэтгэлийг харуулдаг.

```

-cp target/justify-1.0-SNAPSHOT.jar com.example.justify.Appmuu\OneDrive\Documents\3A\Algorithm-ii
n-shinjlgee-ba-zohiomj\Bie daalt 2\java>
Row width:30
Choose algorithm: DP -> 1, greedy -> 2
> 1
Enter your text (end with a blank line):
Эрт урьд цагт энэ дэлхийд долоон нар гараад, ган гачиг болоод, газрын хөрс улайдаад, ус мөрөн ширг
ээд, ургамал мод хатаад, амьтан хүн халууцаад, адуу мал харангадаад, байх суухын аргагүй болж гэнэ
. Тэгтэл тэр нутагт Эрхий мэргэн гэдэг харснаа харвадаг, харваснаа онодог хавтай мэргэн харваач ба
йж гэнээ. Цуутын цагаагч гүү цолмон цоохор унага, далан хоёр ижил сүрэг, дархан хул азаргатайгаа үз
эсгэлэнт ногоон хөндийдөө идээшлэн бэлчдэг байжээ.

Result:
Эрт урьд цагт энэ дэлхийд
долоон нар гараад, ган гачиг
болоод, газрын хөрс улайдаад,
ус мөрөн ширгээд, ургамал мод
хатаад, амьтан хүн халууцаад,
адуу мал харангадаад, байх
суухын аргагүй болж гэнэ.
Тэгтэл тэр нутагт Эрхий
мэргэн гэдэг харснаа харвадаг,
харваснаа онодог хавтай
мэргэн харваач байж гэнээ.
Цуутын цагаагч гүү цолмон
цоохор унага, далан хоёр ижил
сүрэг, дархан хул азаргатайгаа
үзэсгэлэнт ногоон хөндийдөө
идээшлэн бэлчдэг байжээ.

Execution time: 2.992 ms
Used memory: 68.6 KiB

```

Зураг 2. Программын ажиллагаа Java-DP algorithm

4.2 Python хувилбарын гаралт

Алгоритм ажилласны дараа бичвэр мөр тус бүрээр дараах байдлаар хэвлэгдэнэ:

- Баруун ба зүүн талаараа тэгшлэгдсэн мөрүүд.
- Сүүлийн мөр нь зөвхөн зүүн талдаа тэгшлэгдэнэ.
- Хоосон зай зөв хуваарилагдсан эсэхийг хэрэглэгч шууд харах боломжтой.

4.3 Алгоритмын гүйцэтгэлийн мэдээлэл

Зураг 3-т Python хэл дээрх ижил функцтэй программын console гаралтыг үзүүлэв.

1. Хэрэглэгчээс:
 - Мөрийн өргөн: 40
 - алгоритмын сонголт (1 → DP, 2 → greedy, жишээнд 2)
 - Англи, Монгол текст
 - хоосон мөрөөр оролтыг дуусгасан.
2. greedy_justify функц өгөгдсөн мөрийн өргөний дагуу үгсийг мөр болгон тасалж, зүүн талаас тэгшлэн хэвлэж байна.
3. Төгсгөлд:
 - Гүйцэтгэлийн хугацаа: 0.291 ms

- Санах ойн зарцуулалт: 2.0 KiB
гэсэн мэдээлэл нэмэлтээр харагдана.

```
(.venv) PS C:\Users\irmuu\OneDrive\Documents\3A\Algorithm-iin-shinjlgee-ba-zohiomj\Bie daalt 2\python> python .\main.py
Мөрийн өргөн: 40
Алгоритмаа сонгоно уу: DP -> 1, greedy -> 2
> 2
Текстээ оруулна уу (хоосон мөрөөр дуусгана):
Хүүхэд байхаасаа бидний сонсож өссөн Монгол ардын шилдэг 10 үлгэрийн жагсаалтыг танилцуулж байна.
Эдгээр үлгэрүүд нь өнөөг хүртэл ам дамжин яригдсаар ирсэн нь гайхамшигтай биш гэж үү? Монгол ардын
үлгэрүүд нь Монгол соёл, сургамжит байдлыг агуулснаараа гадаад үлгэрүүдээс эрс ялгаатайг хэлэх юу
н.

Үр дүн:
Хүүхэд байхаасаа бидний сонсож өссөн
Монгол ардын шилдэг 10 үлгэрийн
жагсаалтыг танилцуулж байна. Эдгээр
үлгэрүүд нь өнөөг хүртэл ам дамжин
яригдсаар ирсэн нь гайхамшигтай биш гэж
үү? Монгол ардын үлгэрүүд нь Монгол
соёл, сургамжит байдлыг агуулснаараа
гадаад үлгэрүүдээс эрс ялгаатайг хэлэх
юун.

Гүйцэтгэлийн хугацаа: 0.291 ms
Санах ойн зарцуулалт: 2.0 KiB
```

Зураг 3. Программын ажиллагаа Python-Greedy

4.4 Алгоритмын гүйцэтгэлийн мэдээлэл

Бичвэрийн доод талд алгоритмын ажиллагааны дараах үзүүлэлтүүдийг харуулна:

- Гүйцэтгэлийн хугацаа (Execution time, ms)
- Санах ойн зарцуулалт (Memory used, KiB)

Эдгээрийг систем Runtime болон ThreadMXBean ашиглан тооцдог бөгөөд Greedy ба DP алгоритмын гүйцэтгэлийг харьцуулахад чухал мэдээлэл болдог.

5. АЛГОРИТМЫН ГҮЙЦЭТГЭЛИЙН ШИНЖИЛГЭЭ

Энэ хэсэгт бичвэр жигдрүүлэх Greedy болон Dynamic Programming алгоритмуудыг Java, Python хэл дээр ижил оролттой нөхцөлд ажиллуулж, **гүйцэтгэлийн хугацаа** болон **санах ойн хэрэглээг** хэмжсэн үр дүнг харьцуулан шинжилнэ.

Шинжилгээний зорилго нь онолын төвөгшил (Greedy – $O(n)$, DP – $O(n^2)$) ба бодит туршилтын үр дүн хэр зэрэг нийцэж буйг тодорхойлох юм.

5.1 Туршилтын орчны мэдээлэл

Доорх хүснэгтэд туршилт хийсэн орчин болон тохиргооны үндсэн мэдээллийг нэгтгэв.

Үзүүлэлт	Утга
----------	------

Хөгжүүлэлтийн орчин	VSCode
OS	Windows 11 (64-bit)
RAM	16 GB
Процессор	Intel Core i5-13450HX (2.4 GHz)
Java хувилбар	OpenJDK 21
Python хувилбар	Python 3.12.2
Мөрийн өргөн (row width)	50
Оролтын текст	Монгол үсэг – 169 үг, 1109 тэмдэгт, Англи үсэг – 169 үг, 1040 тэмдэгт
Туршилтын тоо	алгоритм бүрийг 5 удаа

Хүснэгт 4. Орчны үзүүлэлт

5.2 Туршилтын үр дүнгийн хүснэгт

Алгоритм бүрийг нэг ижил текст, нэг ижил мөрийн өргөнөөр олон удаа ажиллуулж, тухайн үед:

- Ажиллах хугацаа (ms)
- Санах ойн хэрэглээ (KiB эсвэл MB)

зэргийг хэмжинэ.

Алгоритм	Хэл	Туршилт №	Хугацаа (ms)	Санах (KiB/MB) ой	Дундаж хугацаа (ms)	Дундаж санах ой (KiB/MB)
DP	Java	1	6.450	96.5	8.021	96.5
		2	8.144	96.5		
		3	7.902	96.5		
		4	10.834	96.5		
		5	6.779	96.5		
Greedy	Java	1	11.060	77.2	6.436	77.2
		2	4.234	77.2		
		3	5.782	77.2		
		4	5.136	77.2		
		5	5.968	77.2		
DP	Python	1	18.958	29.8	19.121	29.8
		2	22.103	29.8		
		3	13.669	29.8		

Greedy	Python	4	16.872	29.8	1.118	9.2
		5	24.006	29.8		
		1	0.705	9.2		
		2	1.206	9.2		
		3	1.260	9.2		
		4	1.259	9.2		
		5	1.162	9.2		

Хүснэгт 5. Үр дүн

5.3 Харьцуулсан шинжилгээ

Туршилтын дүнгээс харахад Greedy ба DP алгоритмууд, мөн Java ба Python хэлний гүйцэтгэл дараах онцлогуудтай байна.

5.3.1 Greedy vs DP (адил хэл дээр)

Java:

- Greedy алгоритмын дундаж хугацаа 6.436 ms, DP-нийх 8.021 ms байна. Онолын дагуу Greedy нь нэг удаагийн шугаман гүйлттэй учир DP-эс хурдан ажиллаж байгааг баталж байна.
- Санах ойн хэрэглээ: Greedy – 77.2 KiB, DP – 96.5 KiB. DP нь зардлын хүснэгт, таслалтын массив хадгалдаг тул илүү их санах ой ашиглаж байна.

Python:

- Greedy-ийн дундаж хугацаа 1.118 ms, DP-нийх 19.121 ms. Python дээр ч мөн адил Greedy алгоритм маш хурдан, DP алгоритм нэлээд удаан ажилласан.
- Санах ой: Greedy – 9.2 KiB, DP – 29.8 KiB. DP нь Greedy-эс ойролцоогоор 3 дахин их санах ой хэрэглэж байна.

Хоёр хэл дээр хоёулд нь Greedy алгоритм хурд, санах ойн аль алинд DP-эс илүү хэмнэлттэй байна. Харин DP алгоритм нь илүү их нөөц ашиглаж байгаа ч онолын хувьд бичвэрийн жигдрэлтийн чанарыг илүү сайн хангах боломжтой.

5.3.2 Java vs Python (адил алгоритм дээр)

DP алгоритм:

- Java дахь дундаж хугацаа 8.021 ms, Python дахь нь 19.121 ms. DP алгоритм Java дээр 2 орчим дахин хурдан ажилласан.
- Харин санах ой: Java – 96.5 KiB, Python – 29.8 KiB. Java хувилбар илүү хурдтай боловч санах ойг илүү их зарцуулж байна.

Greedy алгоритм:

- Java: 6.436 ms, Python: 1.118 ms. Энэ тохиолдолд эсрэгээрээ Python хувилбар Java-ас мэдэгдэхүйц хурдан гарсан. Энэ нь кодын хэрэгжилт, оролтын хэмжээ, хэмжилтийн нарийвчлал зэрэгтэй холбоотой байж болох бөгөөд Greedy алгоритмын тогтмол үржигдэхүүн (constant factor) Python дээр бага байгааг харуулж байна.
- Санах ой: Java – 77.2 KiB, Python – 9.2 KiB. Python Greedy хувилбар Java-ас илүү бага санах ой ашиглаж байна.

DP алгоритмын хувьд Java илүү хурдан, Greedy алгоритмын хувьд энэ туршилтаар Python хурдан гарсан ч хоёр хэлний санах ойн хэрэглэлийг харьцуулахад Python нийтдээ бага нөөц ашигласан байна. Хэл сонгохдоо хэрэгжүүлэлтийн онцлог, платформын шаардлага, мөн гүйцэтгэлийн чиг баримжааг хамтад нь харгалзах шаардлагатай.

6. ТЕСТ БА БАТАЛГААЖУУЛАЛТ

6.1 Зорилго

Энэхүү хэсгийн зорилго нь бичвэр жигдрүүлэх Greedy болон Dynamic Programming алгоритмуудын үнэн зөв ажиллагаа, форматын чанар, мөн онолын шинж чанаруудыг баталгаажуулах юм.

Тестүүдээр дараахыг шалгав:

- Мөрийн урт нь өгөгдсөн өргөнөөс хэтрэхгүй, сүүлийн мөрөөс бусад нь яг ижил өргөнтэй байх.
- Жигдрүүлсний дараа үгсийн дараалал, хоорондын зайны логик алдагдаагүй байх (үг нэмэгдэх/алдахгүй).
- DP алгоритмын гаргасан шийдлийн badness (хоосон зайны зардал) нь Greedy алгоритмынхаас ихгүй байх, өөрөөр хэлбэл онолын хувьд илүү оновчтой эсвэл дүйцэхүйц байгааг батлах.

6.2 JustifyTest.java — Нэгж тестийн баталгаа

Java талын тестүүдийг JustifyTest.java файлд бичсэн. Энэ файл нь DPJustifier болон GreedyJustifier классуудын үйлдлийг жижиг хэмжээний жишээнүүд дээр шалгана.

Үндсэн шалгалтууд:

Мөрийн урт болон жигдрэлт

- Java Greedy болон DP функцээр буцаасан мөр бүрийн урт тухайн мөрийн өргөнтэй тэнцүү, сүүлийн мөрийн хувьд $\leq$ өргөн байгааг шалгана.

Үгсийн хадгалагдалт

- Оролтын үгсийг нэг мөр болгон нийлүүлж, гаралтын мөрүүдээс whitespace-ийг цэвэрлэж дахин нэг мөр болгосон үр дүнтэй харьцуулна. Ингэснээр алгоритм үг хасаагүй, солиогүй, эсвэл илүү үг оруулаагүй гэдгийг баталдаг.

Greedy / DP-гийн үндсэн case-үүд

- Англи болон монгол текстийг жигдрүүлэхэд мөрийн урт алдагдахгүй, зөв таслал хийж байгааг шалгана.
- Нэг урт үг, яг таарч багтах мөр зэрэг захын тохиолдлууд дээр Greedy ба DP хоёр ижил зөв үр дүн өгч буй эсэхийг баталгаажуулна.

Ийнхүү Java талын нэгж тестүүд нь алгоритм бүрийн үндсэн форматлах логик болон өгөгдлийн алдагдлыг шалгаж, консоль дээрх жигдлэлт онолтой нийцэж байгааг баталгаажуулсан.

6.3 test_justify.py — Нэгж тест ба онолын баталгаа

Python талын тестүүдийг test_justify.py файлд unittest модулиор бичсэн бөгөөд greedy_justify, dp_justify функцүүдийн ажиллагааг илүү нарийвчлалтай шалгана.

6.3.1 Үндсэн бүтэц ба нийтлэг шалгалтууд

Файл дотор TestJustify нэртэй тест класс тодорхойлогдож, түүнд туслах дараах хоёр функц байдаг:

`_common_checks(lines, width)`

- Сүүлийн мөрөөс бусад бүх мөрийн урт яг width-тай тэнцүү, сүүлийн мөрийн урт $\leq$ width байх ёстойг шалгана.

`_words_preserved(lines, original_words)`

- Оролтын үгсийг нэг мөр болгон холбож, гаралтын мөрүүдээс илүүдэл хоосон зайг арилган нэг мөр болгоод харьцуулна.
- Ингэснээр Greedy болон DP аль аль нь үгсийн дараалал, тоог өөрчлөөгүй гэдгийг батална.

test_greedy_basic, test_dp_basic тестүүд дээр эдгээр туслах функцүүдийг ашиглан англи хэл дээрх жишээ бичвэрийг жигдрүүлэх үед Greedy болон DP аль аль нь зөв ажиллаж байгааг баталдаг.

6.3.2 Монгол текст ба badness-ийн харьцуулалт

test_mongolian_unicode тест нь Монгол хэл дээрх Unicode бичвэр дээр Greedy, DP алгоритмуудыг зэрэг туршина.

- Эхлээд хоёр алгоритмаар гаргасан мөрүүдийн уртыг _common_checks функцээр шалгана.
- Дараа нь тусгай badness(lines, width) туслах функц ашиглан DP-ээр гаргасан жигдрэлтийн зардал нь Greedy-гийнхээс ихгүй буюу $\text{badness(dp)} \leq \text{badness(greedy)}$ байх ёстойг assert хийдэг.
- Ингэснээр DP алгоритм нь онолын дагуу “зардлын функцийг багасгасан” буюу Greedy-гэс дор хаяж илүү муу биш шийдэл олж байгааг тестийн төвшинд баталгаажуулж байна.

Мөн `_words_preserved` функцээр DP хувилбар Монгол текстийн үгсийг бүрэн хадгалж байгааг шалгана.

6.3.3 Захын тохиолдлууд

Дараах нэмэлт тестүүд нь алгоритмын захын нөхцөл дээр зөв ажиллаж буй эсэхийг батална:

`test_exact_fit_line`

- Нийт урт нь мөрийн өргөнтэй яг тэнцүү хоёр үгтэй жишээнд Greedy, DP аль аль нь нэмэлт хоосон зайгүй, нэг мөр хэвлэж байгааг шалгана.

`test_single_long_word`

- Мөрийн өргөнөөс урт ганц үг өгөгдөхөд Greedy болон DP функц хоёул нэг мөрт тэр үгийг бүтнээр нь буцааж байгааг батална (таслалгүй).

`test_two_words_vs_single_word_penalty`

- `["aaaaa", "bbbbb", "cc"]` гэх үгсийг өргөн 11-т DP алгоритмаар жигдрүүлэхэд нийлбэр badness хамгийн бага байх шийдэл болох `"aaaaa bbbbbb"` мөрийг агуулж байх ёстойг assert хийнэ.
- Энэ нь DP алгоритм зардлын функц дээр тулгуурлан хоёр мөрийн хувилбаруудаас оновчтойг сонгож буйг харуулна.

7. ДҮГНЭЛТ

Энэхүү бие даалтын хүрээнд бичвэрийг мөрийн өргөнд тааруулан зүүн ба баруун талаараа жигдрүүлэх Greedy болон Dynamic Programming алгоритмуудыг судалж, Java болон Python хэл дээр хэрэгжүүлэн туршив. Хоёр алгоритм нь бичвэрийн таслалт, хоосон зай хуваарилалтын зарчим болон гүйцэтгэлийн үр дүнгээрээ ялгаатай болох нь практик туршилтаар батлагдлаа.

Greedy алгоритм нь шугаман төвөгшилтэй тул хурд болон санах ойн зарцуулалтын хувьд илт давуу ажиллаж, их хэмжээний өгөгдлийг хөнгөхөн боловсруулах чадвартай байв. Харин Dynamic Programming алгоритм нь илүү их нөөц ашигладаг хэдий ч бичвэрийн жигдрэлтийн чанарыг зардлын функцээр оновчлон шийддэг давуу талтайг туршилтын badness хэмжилтүүд харуулсан.

Java болон Python орчинд хийсэн туршилтаар Greedy алгоритм хоёр хэл дээр тогтвортой хөнгөн ажилласан бол DP алгоритмын гүйцэтгэл хэл тус бүрд ялгаатай байв. Java орчинд DP илүү хурдтай байсан бол Python орчинд санах ой хэмнэлттэй ажилласан нь хэлний онцлог, хэрэгжилтийн бүтэцтэй холбоотой байлаа.

Ерөнхийд нь дүгнэхэд, энгийн хэрэглээнд Greedy алгоритм практик хувьд илүү тохиромжтой, харин бичвэрийн жигдрэлтийн чанар болон үзэмжийг чухалчилсан тохиолдолд Dynamic Programming алгоритм илүү оновчтой шийдэл болохыг энэхүү судалгаа харууллаа.

8. НОМ ЗҮЙ

- [1] "<https://download.geofabrik.de/asia/mongolia.html>," 18 10 2025. [Online]. Available: <https://download.geofabrik.de/asia/mongolia.html>.
- [2] "plantuml," 4 August 2021. [Online]. Available: <https://www.plantuml.com/>.
- [3] "spellcheck," gov, 1 1 2025. [Online]. Available: <https://spellcheck.mn/>.
- [4] "Greedy Algorithms," 11 7 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/greedy-algorithms/>.
- [5] "Dynamic Programming or DP," 3 10 2025. [Online]. Available: <https://www.geeksforgeeks.org/competitive-programming/dynamic-programming/>.
- [6] Reducible, "Hunspell," 6 July 2020. [Online]. Available: <https://en.wikipedia.org/wiki/Hunspell>.